Date: **Roll No.:** | | | | | | | | | | |

UNIT-1

Practice 1: Importing and Exploring Datasets

AIM:

To load a dataset using Python libraries and perform basic data exploration to understand its structure and summary.

DESCRIPTION:

In this experiment, we learn how to import datasets into Python using the `pandas` library and explore them to gain basic insights. Data exploration involves understanding the dataset's shape, column names, data types, and statistical properties. These steps form the foundation for any data analysis process.

PROGRAM:

```
# Importing necessary library
import pandas as pd

# Loading the dataset (example: students.csv)
data = pd.read_csv("students.csv")

# Display first 5 rows of the dataset
print("First 5 Rows of Dataset:")
print(data.head())

# Display the number of rows and columns
print("\nShape of the Dataset:")
print(data.shape)

# Display column names
print("\nColumn Names:")
print(data.columns)

# Display statistical summary of numerical columns
print("\nStatistical Summary:")
print(data.describe())

# Check for missing values
print("\nMissing Values in Each Column:")
print(data.isnull().sum())
```

ADITYA UNIVERSITY

Date: **Roll No.:**

OUTPUT:

```
First 5 Rows of Dataset:
   Name  Age  Marks  Grade
0  Ram   19     85     A
1  Sita  18     92     A+
2  Ravi  20     78     B
3  Rani  19     88     A
4  John  18     67     C

Shape of the Dataset:
(20, 4)

Column Names:
Index(['Name', 'Age', 'Marks', 'Grade'], dtype='object')

Statistical Summary:
             Age      Marks
count  20.000000  20.000000
mean   18.950000  82.150000
std     0.911043   9.257367
min    18.000000  67.000000
max    20.000000  95.000000

Missing Values in Each Column:
Name     0
Age      0
Marks    0
Grade    0
dtype: int64
```

ADITYA UNIVERSITY

Date: **Roll No.:** | | | | | | | | | | |

Practice 2: Basic Data Structures - Working with data structures such as arrays, lists, data frames, and matrices.

AIM:

To understand and work with different data structures such as arrays, lists, data frames, and matrices in Python.

DESCRIPTION:

Data structures are used to store and organize data efficiently. In Python, lists and arrays handle simple collections, while DataFrames and matrices allow structured and numerical data manipulation.

- **Lists** are mutable and store heterogeneous data.
- **Arrays** (from NumPy) store homogeneous numerical data efficiently.
- **DataFrames** (from pandas) store tabular data.
- **Matrices** are 2D numerical structures used for mathematical computations.

PROGRAM:

```
# Importing necessary libraries
import numpy as np
import pandas as pd

# 1. List
marks = [85, 90, 78, 88]
print("List of Marks:", marks)

# 2. Array
arr = np.array([1, 2, 3, 4, 5])
print("\nArray Elements:", arr)

# 3. DataFrame
data = {
    'Name': ['Ram', 'Sita', 'Ravi'],
    'Marks': [85, 92, 78]
}
df = pd.DataFrame(data)
print("\nDataFrame:\n", df)

# 4. Matrix
matrix = np.array([[1, 2], [3, 4]])
print("\nMatrix:\n", matrix)
```

ADITYA UNIVERSITY

Date: **Roll No.:**

OUTPUT:

```
List of Marks: [85, 90, 78, 88]

Array Elements: [1 2 3 4 5]

DataFrame:
   Name  Marks
0   Ram     85
1  Sita     92
2  Ravi     78

Matrix:
[[1 2]
 [3 4]]
```

ADITYA UNIVERSITY

Date: **Roll No.:** | | | | | | | | | | |

UNIT-2

Practice 1: Computing Summary Statistics

AIM:

To compute summary statistics such as mean, median, mode, and standard deviation for a sample dataset using Python libraries.

DESCRIPTION:

Statistical measures like **mean**, **median**, **mode**, and **standard deviation** help describe and understand the key characteristics of a dataset.

- **Mean** gives the average value.
- **Median** represents the middle value when data is sorted.
- **Mode** identifies the most frequent value.
- **Standard Deviation** measures how much values deviate from the mean.

These measures are essential for understanding data distribution and variability before performing further analysis.

PROGRAM:

```
# Importing necessary libraries
import pandas as pd
from statistics import mean, median, mode, stdev

# Sample dataset
data = [45, 50, 55, 60, 65, 70, 75, 80, 85, 90]

# Compute summary statistics
mean_val = mean(data)
median_val = median(data)
mode_val = mode(data)
std_dev = stdev(data)

# Display results
print("Sample Data:", data)
print("Mean:", mean_val)
print("Median:", median_val)
print("Mode:", mode_val)
print("Standard Deviation:", std_dev)
```

ADITYA UNIVERSITY

Date: **Roll No.:**

OUTPUT:

```
Sample Data: [45, 50, 55, 60, 65, 70, 75, 80, 85, 90]
Mean: 67.5
Median: 67.5
Mode: 45
Standard Deviation: 15.138251770487823
```

ADITYA
UNIVERSITY

Date: **Roll No.:** | | | | | | | | | | |

Practice 2: Handling Missing Values and Outliers

AIM:

To handle missing values and outliers in a dataset using techniques such as imputation and filtering.

DESCRIPTION:

Real-world datasets often contain **missing values** or **outliers**, which can affect the accuracy of analysis and model performance.

- **Missing values** can be handled using **imputation** techniques such as replacing them with the mean, median, or mode of the column.
- **Outliers** can be handled using **filtering** or **capping** methods (like removing extreme values beyond a certain range).

This experiment demonstrates how to clean and preprocess such data for accurate analysis.

PROGRAM:

```
# Importing necessary libraries
import pandas as pd
import numpy as np

# Creating a sample dataset with missing and outlier values
data = {
    'Marks': [85, 90, np.nan, 95, 100, 30, 110, np.nan, 88, 92]
}
df = pd.DataFrame(data)

print("Original Dataset:")
print(df)

# Handling Missing Values - Imputation with Mean
mean_value = df['Marks'].mean()
df['Marks'].fillna(mean_value, inplace=True)

print("\nAfter Handling Missing Values (Imputed with Mean):")
print(df)

# Handling Outliers - Filtering
# Assuming valid marks are between 0 and 100
filtered_df = df[(df['Marks'] >= 0) & (df['Marks'] <= 100)]

print("\nAfter Removing Outliers:")
print(filtered_df)
```

ADITYA UNIVERSITY

Date: **Roll No.:**

OUTPUT:

```
Original Dataset:
    Marks
0    85.0
1    90.0
2     NaN
3    95.0
4   100.0
5    30.0
6   110.0
7     NaN
8    88.0
9    92.0

After Handling Missing Values (Imputed with Mean):
    Marks
0    85.000000
1    90.000000
2    86.285714
3    95.000000
4   100.000000
5    30.000000
6   110.000000
7    86.285714
8    88.000000
9    92.000000

After Removing Outliers:
    Marks
0    85.000000
1    90.000000
2    86.285714
3    95.000000
4   100.000000
5    30.000000
7    86.285714
8    88.000000
9    92.000000
```

ADITYA UNIVERSITY

Date: **Roll No.:** | | | | | | | | | | |

UNIT-3

Practice 1: Reading Data from a JSON File

AIM:

To read data from a JSON file into a DataFrame using the Pandas library and parse nested JSON structures to convert them into a flat table format.

DESCRIPTION:

JSON (JavaScript Object Notation) is a lightweight data format commonly used for storing and transferring structured data.
Pandas provides the `pd.read_json()` function to load JSON data directly into a DataFrame for easy analysis.
When the JSON contains **nested structures**, it must be **normalized** or flattened using `json_normalize()` to create a tabular format. This makes it easier to work with hierarchical data in analytical workflows.

PROGRAM:

```
# Importing necessary libraries
import pandas as pd
from pandas import json_normalize

# Sample nested JSON data
data = {
    "students": [
        {"name": "Ram", "details": {"age": 20, "marks": 85}},
        {"name": "Sita", "details": {"age": 19, "marks": 90}},
        {"name": "Ravi", "details": {"age": 21, "marks": 78}}
    ]
}

# Convert JSON data into DataFrame
df = json_normalize(data["students"])

# Display the flattened DataFrame
print("Flattened DataFrame from JSON:")
print(df)
```

ADITYA UNIVERSITY

Date: **Roll No.:**

OUTPUT:

```
Flattened DataFrame from JSON:
   name  details.age  details.marks
0   Ram           20             85
1  Sita           19             90
2  Ravi           21             78
```

ADITYA
UNIVERSITY

Date: **Roll No.:** | | | | | | | | | | |

Practice 2: Web Scraping from an HTML Page

AIM:

To perform web scraping on an HTML page using the BeautifulSoup library in Python and extract table data into a Pandas DataFrame.

DESCRIPTION:

Web scraping is the process of extracting data from web pages automatically.
The **BeautifulSoup** library in Python allows parsing of HTML documents to extract specific information, such as tables, lists, or text.
In this experiment, we read an HTML page containing a table, extract the rows and columns using BeautifulSoup, and then store the extracted data into a **DataFrame** for further analysis.

PROGRAM:

```
# Importing required libraries
import requests
from bs4 import BeautifulSoup
import pandas as pd

# Sample HTML content (can also use a webpage URL)
html_data = """
<table>
<tr><th>Name</th><th>Age</th><th>Marks</th></tr>
<tr><td>Ram</td><td>20</td><td>85</td></tr>
<tr><td>Sita</td><td>19</td><td>90</td></tr>
<tr><td>Ravi</td><td>21</td><td>78</td></tr>
</table>
"""

# Parse the HTML data using BeautifulSoup
soup = BeautifulSoup(html_data, 'html.parser')

# Extract table rows
rows = []
for tr in soup.find_all('tr'):
    cols = [td.text for td in tr.find_all(['td', 'th'])]
    rows.append(cols)

# Convert to DataFrame
df = pd.DataFrame(rows[1:], columns=rows[0])

# Display the DataFrame
print("Extracted DataFrame from HTML Table:")
print(df)
```

ADITYA UNIVERSITY

Date: **Roll No.:**

OUTPUT:

```
Extracted DataFrame from HTML Table:
   Name Age Marks
0   Ram  20    85
1  Sita  19    90
2  Ravi  21    78
```

ADITYA
UNIVERSITY

Date: **Roll No.:** | | | | | | | | | | |

UNIT-4

Practice 1: Plotting Histograms and Box Plots

AIM:

To plot histograms and box plots using Python to understand the distribution and spread of a single variable.

DESCRIPTION:

A **Histogram** shows the frequency distribution of numerical data by dividing it into intervals (bins). It helps visualize the **shape**, **center**, and **spread** of the data.
A **Box Plot** (or Whisker Plot) displays the distribution based on five summary statistics — *minimum, first quartile (Q1), median, third quartile (Q3), and maximum*. It also identifies **outliers** easily.
Both plots are essential for **univariate data analysis**, providing a visual summary of data variability and skewness.

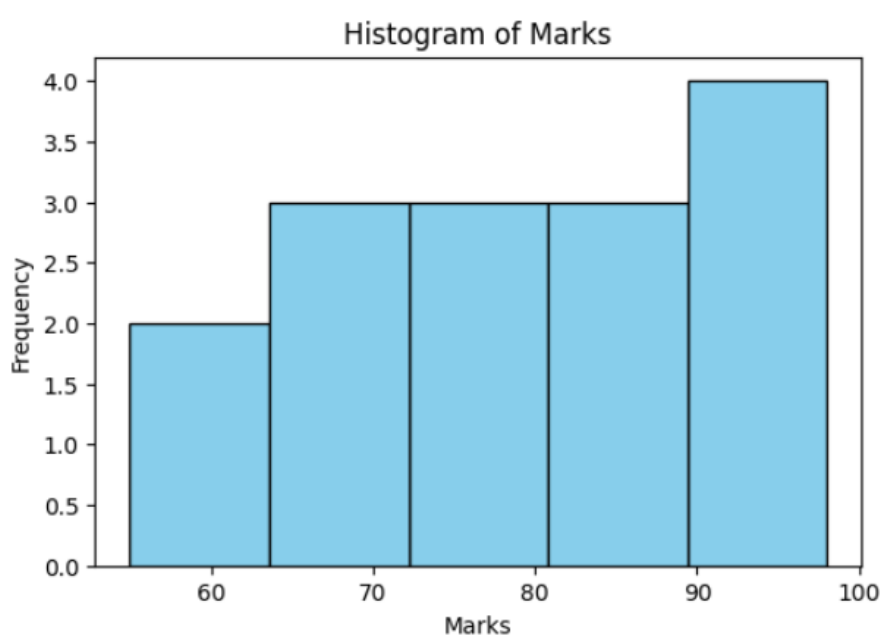
PROGRAM:

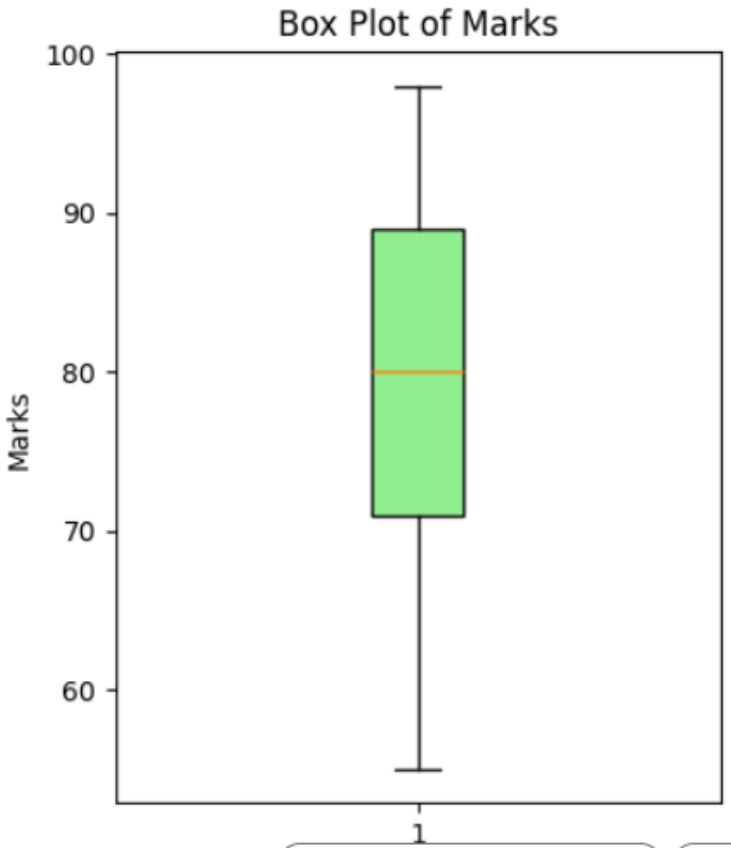
```
# Importing necessary libraries
import pandas as pd
import matplotlib.pyplot as plt

# Sample dataset
data = {'Marks': [55, 60, 65, 70, 72, 74, 78, 80, 82, 85, 88, 90,
92, 95, 98]}
df = pd.DataFrame(data)

# Plotting Histogram
plt.figure(figsize=(6,4))
plt.hist(df['Marks'], bins=5, color='skyblue', edgecolor='black')
plt.title('Histogram of Marks')
plt.xlabel('Marks')
plt.ylabel('Frequency')
plt.show()

# Plotting Box Plot
plt.figure(figsize=(4,5))
plt.boxplot(df['Marks'], patch_artist=True,
boxprops=dict(facecolor='lightgreen'))
plt.title('Box Plot of Marks')
plt.ylabel('Marks')
plt.show()
```

ADITYA UNIVERSITY

Date: **Roll No.:**

OUTPUT:





Date: **Roll No.:** | | | | | | | | | | |

Practice 2: Scatter Plot and Correlation

AIM:

To create scatter plots and calculate correlation coefficients to study the relationship between two variables.

DESCRIPTION:

A **Scatter Plot** visually represents the relationship between two numerical variables by plotting one on the x-axis and the other on the y-axis.
The **Correlation Coefficient** measures the strength and direction of the linear relationship between the two variables.

- If correlation ≈ **+1**, there is a strong positive relationship.
- If correlation ≈ **-1**, there is a strong negative relationship.
- If correlation ≈ **0**, there is little or no linear relationship.

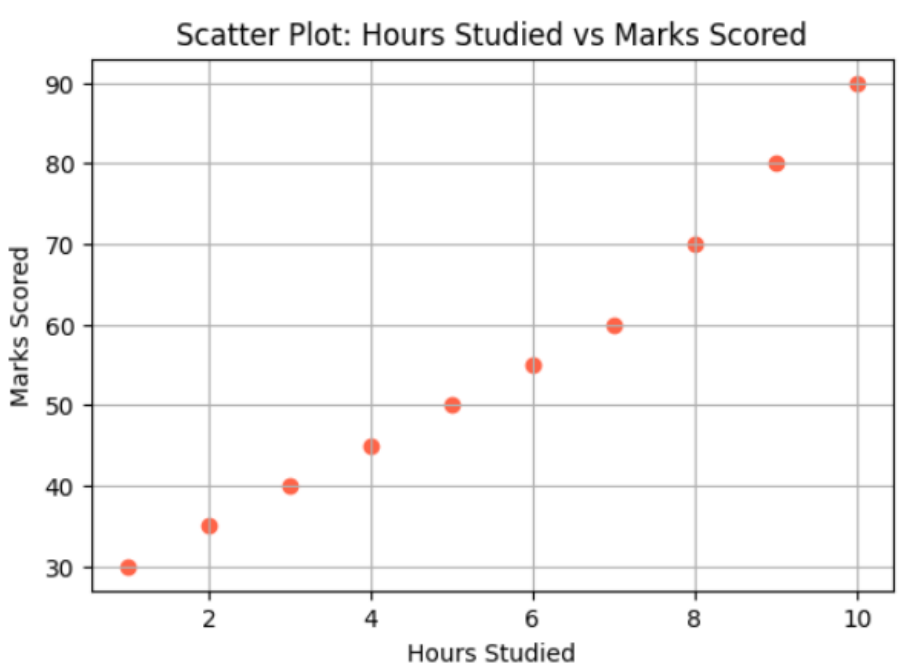
PROGRAM:

```
# Importing necessary libraries
import pandas as pd
import matplotlib.pyplot as plt

# Sample dataset
data = {
    'Hours_Studied': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    'Marks_Scored': [30, 35, 40, 45, 50, 55, 60, 70, 80, 90]
}
df = pd.DataFrame(data)

# Scatter Plot
plt.figure(figsize=(6,4))
plt.scatter(df['Hours_Studied'], df['Marks_Scored'],
color='tomato')
plt.title('Scatter Plot: Hours Studied vs Marks Scored')
plt.xlabel('Hours Studied')
plt.ylabel('Marks Scored')
plt.grid(True)
plt.show()

# Calculate Correlation Coefficient
corr = df['Hours_Studied'].corr(df['Marks_Scored'])
print("Correlation Coefficient:", corr)
```

ADITYA UNIVERSITY

Date: **Roll No.:**

OUTPUT:




```
Correlation Coefficient: 0.9855402424871892
```

```
Correlation Coefficient: 0.985
```

ADITYA UNIVERSITY

Date: **Roll No.:** | | | | | | | | | | |

UNIT-5

Practice 1: Managing and Querying Graph Databases using Neo4j

AIM:

To get hands-on experience with Neo4j and understand how to manage and query graph databases.

DESCRIPTION:

Neo4j is a popular **graph database management system** that stores data in the form of **nodes**, **relationships**, and **properties**.
Unlike relational databases, Neo4j focuses on the relationships between data, making it ideal for social networks, recommendation systems, and fraud detection.

In this experiment, we will:

- Create nodes (entities)
- Define relationships between nodes
- Retrieve and visualize data from a graph structure using **Cypher Query Language (CQL)**.

PROGRAM (Cypher Queries):

```
// 1. Create Nodes
CREATE (a:Person {name: 'Alice', age: 25});
CREATE (b:Person {name: 'Bob', age: 27});
CREATE (c:Person {name: 'Charlie', age: 30});

// 2. Create Relationships
CREATE (a)-[:FRIENDS_WITH]->(b);
CREATE (b)-[:FRIENDS_WITH]->(c);
CREATE (a)-[:COLLEAGUE_OF]->(c);

// 3. Retrieve All Nodes
MATCH (n) RETURN n;

// 4. Retrieve All Relationships
MATCH (a)-[r]->(b) RETURN a, r, b;

// 5. Display Specific Relationship
MATCH (a:Person)-[:FRIENDS_WITH]->(b:Person)
RETURN a.name AS Person1, b.name AS Person2;
```

ADITYA UNIVERSITY

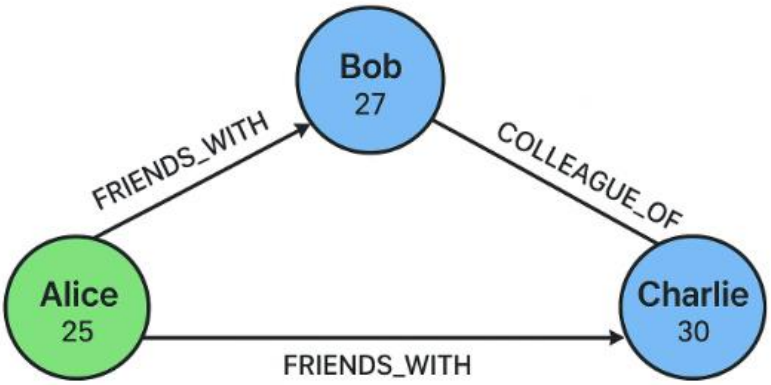
Date: **Roll No.:**

EXPECTED OUTPUT:

```
Person1      Person2
-------------------
Alice        Bob
Bob          Charlie
```

Graph Visualization:

- Three nodes appear: **Alice**, **Bob**, and **Charlie**.
- Arrows (relationships) show:
  - Alice → Bob (FRIENDS_WITH)
  - Bob → Charlie (FRIENDS_WITH)
  - Alice → Charlie (COLLEAGUE_OF)



Date: **Roll No.:**

Practice 2 : Writing Cypher Queries in Neo4j

AIM:

To practice writing Cypher queries to interact with and analyze graph data stored in Neo4j.

DESCRIPTION:

Cypher Query Language (CQL) is the declarative language used in Neo4j to query and manipulate graph data.
It allows users to:

- **Create** and **update** nodes and relationships
- **Filter** and **match patterns**
- **Aggregate** and **analyze** data relationships

This experiment focuses on writing Cypher queries to extract meaningful insights from graph data.

PROGRAM (Cypher Queries):

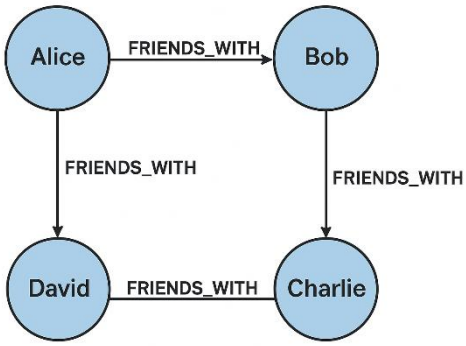
```
// 1. Find all people older than 26
MATCH (p:Person)
WHERE p.age > 26
RETURN p.name, p.age;

// 2. Count how many FRIENDS_WITH relationships exist
MATCH (:Person)-[r:FRIENDS_WITH]->(:Person)
RETURN COUNT(r) AS TotalFriendships;

// 3. Find friends of a particular person (Alice)
MATCH (a:Person {name: 'Alice'})-[:FRIENDS_WITH]->(friends)
RETURN friends.name AS FriendsOfAlice;

// 4. Find mutual connections
MATCH (a:Person)-[:FRIENDS_WITH]->(b:Person)-[:FRIENDS_WITH]-
>(c:Person)
RETURN a.name AS Person, c.name AS MutualConnection;
```

ALL INDIA UNIVERSITY

Date: **Roll No.:**

OUTPUT:

```
p.name      p.age
-----------------
Bob         27
Charlie     30
TotalFriendships
----------------
2
FriendsOfAlice
---------------
Bob
Person      MutualConnection
----------------------------
Alice       Charlie
```



ADITYA UNIVERSITY